

CAS4160 Homework 2: Policy Gradients

[Your name]
[Your student ID]

1 Introduction

이 과제의 목표는 정책 경사 방법(Policy Gradient Methods)을 이해하는 데 도움을 주고, 보상-to-go(reward-to-go), 신경망 베이스라인(neural network baselines), Generalized Advantage Estimator(GAE)와 같은 분산 감소 기법(variance reduction tricks)을 포함하며, 마지막으로 실제로 매우 잘 작동하는 널리 사용되는 온-폴리시 RL 알고리즘인 Proximal Policy Optimization (PPO)을 구현하는 것입니다. 여기에 Overleaf의 이 과제 보고서 템플릿 링크가 있습니다.

2 Review

2.1 Policy Gradient

강화학습의 목적은 목적 함수(objective function)를 최대화하는 θ 를 학습하는 것임을 상기하라:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}(\tau)}[r(\tau)], \quad (1)$$

각 롤아웃(rollout) τ 의 길이가 T 인 경우, 목적 함수는 다음과 같다:

$$\pi_{\theta}(\tau) = p(\mathbf{s}_0, \mathbf{a}_0, \dots, \mathbf{s}_{T-1}, \mathbf{a}_{T-1}, \mathbf{s}_T) = p(\mathbf{s}_0) \prod_{t=0}^{T-1} \pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t) p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t),$$

and

$$r(\tau) = r(\mathbf{s}_0, \mathbf{a}_0, \dots, \mathbf{s}_{T-1}, \mathbf{a}_{T-1}) = \sum_{t=0}^{T-1} r(\mathbf{s}_t, \mathbf{a}_t).$$

NOTE: 이 과제에서는 t 가 1이 아닌 0부터 시작한다. 이는 구현하기 더 쉽다.

정책 경사 접근법은 이 목적 함수의 그래디언트를 직접 계산하는 것이다:

$$\begin{aligned} \nabla_{\theta} J(\theta) &= \nabla_{\theta} \mathbb{E}_{\tau \sim \pi_{\theta}(\tau)}[r(\tau)] \\ &= \nabla_{\theta} \int \pi_{\theta}(\tau) r(\tau) d\tau \\ &= \int \pi_{\theta}(\tau) \nabla_{\theta} \log \pi_{\theta}(\tau) r(\tau) d\tau \quad (\because \nabla_{\theta} \pi_{\theta}(\tau) = \pi_{\theta}(\tau) \nabla_{\theta} \log \pi_{\theta}(\tau)) \\ &= \mathbb{E}_{\tau \sim \pi_{\theta}(\tau)} [\nabla_{\theta} \log \pi_{\theta}(\tau) r(\tau)]. \end{aligned}$$

실제로는, 궤적(trajecory) τ 에 대한 기댓값(expectation)은 N 개의 샘플링된 궤적으로 구성된 배치(batch)로부터 근사할 수 있다:

$$\begin{aligned} \nabla_{\theta} J(\theta) &\approx \frac{1}{N} \sum_{i=1}^N \nabla_{\theta} \log \pi_{\theta}(\tau_i) r(\tau_i) \\ &= \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_{it} | \mathbf{s}_{it}) \right) \left(\sum_{t=0}^{T-1} r(\mathbf{s}_{it}, \mathbf{a}_{it}) \right). \end{aligned} \quad (2)$$

여기서 우리는 정책(policy) π_θ 이 상태(state)에 조건화된 행동 공간(action space) 위의 확률 분포임을 알 수 있다. 에이전트-환경 루프(agent-environment loop)에서, 에이전트는 $\pi_\theta(\cdot | \mathbf{s}_t)$ 로부터 행동 $\mathbf{a}_t$ 를 샘플링하고, 환경은 보상 $r(\mathbf{s}_t, \mathbf{a}_t)$ 으로 응답한다.

2.2 Variance Reduction

2.2.1 Reward-to-go

정책 경사의 분산을 줄이는 한 가지 방법은 인과성(causality)을 활용하는 것이다. 즉, 정책은 과거의 보상에는 영향을 미칠 수 없다는 개념이다. 이는 다음과 같은 수정된 목적 함수를 도출한다. 여기서 보상의 합은 정책이 쿼리 되는 시점 이전에 얻은 보상은 포함하지 않는다. 이 보상의 합은 Q 함수의 샘플 추정치이며, “reward-to-go”라고 불린다:

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(\mathbf{a}_{it} | \mathbf{s}_{it}) \underbrace{\left(\sum_{t'=t}^{T-1} r(\mathbf{s}_{it'}, \mathbf{a}_{it'}) \right)}_{\text{reward-to-go}}. \quad (3)$$

2.2.2 Discounting

보상에 할인 계수(discount factor) γ 를 곱하는 것은 에이전트가 시간적으로 더 가까운 보상에 더 집중하고, 먼 미래의 보상에는 덜 집중하도록 강려하는 것으로 해석될 수 있다. 이는 또한 먼 미래를 고려할 때 더 많은 분산(variance)과 불확실성이 있을 수 있기 때문에, 분산을 줄이는 방법으로도 생각할 수 있다.

전체 궤적(full trajectory)의 보상에 할인을 적용할 수 있다. Equation (2):

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(\mathbf{a}_{it} | \mathbf{s}_{it}) \right) \left(\sum_{t'=0}^{T-1} \gamma^{t'-1} r(\mathbf{s}_{it'}, \mathbf{a}_{it'}) \right). \quad (4)$$

“reward-to-go”에도 할인을 적용할 수 있다. Equation (3):

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(\mathbf{a}_{it} | \mathbf{s}_{it}) \left(\sum_{t'=t}^{T-1} \gamma^{t'-t} r(\mathbf{s}_{it'}, \mathbf{a}_{it'}) \right). \quad (5)$$

2.2.3 Baseline

또 다른 분산 감소 방법은 보상의 합에서 (현재 상태 $\mathbf{s}_t$ 에만 조건화된) 베이스라인(baseline)을 빼는 것이다:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta(\tau)} \left[\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(\mathbf{a}_t | \mathbf{s}_t) [r(\tau) - b(\mathbf{s}_t)] \right]. \quad (6)$$

이는 정책 경사가 편향되지 않도록 만든다. 왜냐하면

$$\begin{aligned} \mathbb{E}_{\tau \sim \pi_\theta(\tau)} [\nabla_\theta \log \pi_\theta(\mathbf{a}_t | \mathbf{s}_t) b(\mathbf{s}_t)] &= \mathbb{E}_{\tau \sim \pi_\theta(\tau)} [\nabla_\theta \log \pi_\theta(\mathbf{a}_t | \mathbf{s}_t)] \mathbb{E}_{\tau \sim \pi_\theta(\tau)} [b(\mathbf{s}_t)] \\ &= 0 \cdot \mathbb{E}_{\tau \sim \pi_\theta(\tau)} [b(\mathbf{s}_t)] = 0. \end{aligned}$$

이 과제에서는 상태에 의존하는 베이스라인(state-dependent baseline) 역할을 하는 가치 함수(value function) $V_\phi^\pi(\mathbf{s}_t)$ 를 구현할 것이다. 이 가치 함수는 특정 상태에서 시작하여 미래에 받을 보상의 합을 근사하도록 학습된다:

$$V_\phi^\pi(\mathbf{s}_t) \approx \sum_{t'=t}^{T-1} \mathbb{E}_{\tau \sim \pi_\theta(\tau)} [r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) | \mathbf{s}_t]. \quad (7)$$

그러면, 근사된 정책 경사는 이제 다음과 같이 된다:

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_{it} | \mathbf{s}_{it}) \left(\underbrace{\left(\sum_{t'=t}^{T-1} \gamma^{t'-t} r(\mathbf{s}_{it'}, \mathbf{a}_{it'}) \right)}_{\text{reward-to-go}} - \underbrace{V_{\phi}^{\pi}(\mathbf{s}_{it})}_{\text{baseline}} \right). \quad (8)$$

2.2.4 Generalized Advantage Estimator (GAE)

이전 정책 경사 식에서 (명확하게 하기 위해 인덱스 i 를 제거한) 양 $\left(\sum_{t'=t}^{T-1} \gamma^{t'-t} r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) \right) - V_{\phi}^{\pi}(\mathbf{s}_t)$ 은 어드밴티지 함수(advantage function)의 추정치로 해석될 수 있다:

$$A^{\pi}(\mathbf{s}_t, \mathbf{a}_t) = Q^{\pi}(\mathbf{s}_t, \mathbf{a}_t) - V^{\pi}(\mathbf{s}_t), \quad (9)$$

여기서 $Q^{\pi}(\mathbf{s}_t, \mathbf{a}_t)$ 는 Monte Carlo 리턴(Monte Carlo returns)을 사용하여 추정되고, $V^{\pi}(\mathbf{s}_t)$ 는 학습된 가치 함수 V_{ϕ}^{π} 를 사용하여 추정된다. 우리는 또한 Monte Carlo 리턴 대신 V_{ϕ}^{π} 를 사용하여 어드밴티지 함수를 다음과 같이 추정함으로써 분산을 더 줄일 수 있다:

$$A^{\pi}(\mathbf{s}_t, \mathbf{a}_t) \approx \delta_t = r(\mathbf{s}_t, \mathbf{a}_t) + \gamma V_{\phi}^{\pi}(\mathbf{s}_{t+1}) - V_{\phi}^{\pi}(\mathbf{s}_t), \quad (10)$$

(단, 경계 경우(edge case)로 $\delta_{T-1} = r(\mathbf{s}_{T-1}, \mathbf{a}_{T-1}) - V_{\phi}^{\pi}(\mathbf{s}_{T-1})$ 이다.) 그러나 이는 V_{ϕ}^{π} 의 모델링 오류(modeling errors)로 인해 정책 경사 추정치에 편향(bias)을 도입하는 대가를 치른다. 대신 우리는 n -스텝 Monte Carlo 리턴과 V_{ϕ}^{π} 를 조합하여 어드밴티지 함수를 다음과 같이 추정할 수 있다:

$$A_n^{\pi}(\mathbf{s}_t, \mathbf{a}_t) = \sum_{t'=t}^{t+n} \gamma^{t'-t} r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) + \gamma^n V_{\phi}^{\pi}(\mathbf{s}_{t+n+1}) - V_{\phi}^{\pi}(\mathbf{s}_t). \quad (11)$$

n 을 증가시키면 어드밴티지 추정치에서 Monte Carlo 리턴을 더 많이 반영하게 되어 편향(bias)은 낮아지고 분산(variance)은 증가한다. 반대로 n 을 감소시키면 그 반대가 된다. 참고로 $n = T - t - 1$ 로 하면 (13)에서 사용된 편향되지 않았지만 분산이 높은 Monte Carlo 어드밴티지 추정치를 회복하고, $n = 0$ 으로 하면 분산은 낮지만 편향이 높은 어드밴티지 추정치 δ_t 를 회복한다.

우리는 여러 n -스텝 어드밴티지 추정치를 지수 가중 평균(exponentially weighted average)으로 결합할 수 있는데, 이를 Generalized Advantage Estimator (GAE)라고 한다. $\lambda \in [0, 1]$ 이라고 하자. 그러면 다음과 같이 정의한다:

$$A_{GAE}^{\pi}(\mathbf{s}_t, \mathbf{a}_t) = \frac{1 - \lambda^{T-t-1}}{1 - \lambda} \sum_{n=1}^{T-t-1} \lambda^{n-1} A_n^{\pi}(\mathbf{s}_t, \mathbf{a}_t), \quad (12)$$

여기서 $\frac{1 - \lambda^{T-t-1}}{1 - \lambda}$ 는 정규화 상수(normalizing constant)이다. 참고로 λ 값이 클수록 더 큰 n 값을 가진 어드밴티지 추정치를 강조하고, λ 값이 작을수록 그 반대가 된다. 따라서 λ 는 편향-분산 트레이드오프(bias-variance tradeoff)를 조절하는 역할을 한다. λ 를 증가시키면 편향은 감소하고 분산은 증가한다. 무한 시간 지평(infinite horizon) 경우($T = \infty$)에서는 다음과 같이 나타낼 수 있다:

$$\begin{aligned} A_{GAE}^{\pi}(\mathbf{s}_t, \mathbf{a}_t) &= \frac{1}{1 - \lambda} \sum_{n=1}^{\infty} \lambda^{n-1} A_n^{\pi}(\mathbf{s}_t, \mathbf{a}_t) \\ &= \sum_{t'=t}^{\infty} (\gamma \lambda)^{t'-t} \delta_{t'}, \end{aligned}$$

여기서는 간결함을 위해 유도 과정(derivation)을 생략하였다 (자세한 내용은 [GAE 논문](#)을 참조하라). 유한 시간 지평(finite horizon) 경우에는 다음과 같이 쓸 수 있다:

$$A_{GAE}^{\pi}(\mathbf{s}_t, \mathbf{a}_t) = \sum_{t'=t}^{T-1} (\gamma\lambda)^{t'-t} \delta_{t'}, \quad (13)$$

이는 Generalized Advantage Estimator (GAE)를 효율적으로 구현할 수 있는 방법이 된다. 왜냐하면 우리는 다음과 같이 재귀적으로 계산할 수 있기 때문이다:

$$A_{GAE}^{\pi}(\mathbf{s}_t, \mathbf{a}_t) = \delta_t + \gamma\lambda A_{GAE}^{\pi}(\mathbf{s}_{t+1}, \mathbf{a}_{t+1}) \quad (14)$$

3 Code Structure Overview

훈련은 스크립트 `run_hw2.py`에서 시작된다. 이 스크립트에는 훈련(training), 평가(evaluation), 그리고 로깅(logging)이 이루어지는 함수 `run_training_loop`가 포함되어 있다.

에이전트는 `run_hw2.py`에도 정의되어 있으며, `cas4160/agents/pg_agent.py`에 있는 `PGAgent`의 인스턴스이다. `PGAgent` 클래스는 두 개의 주요 네트워크를 변수로 가지고 있다:

- **actor** 네트워크: `cas4160/networks/policies.py`에 있는 `MLPPolicyPG`. 관찰(observations)을 입력으로 받아 행동(actions)을 출력한다.
- **critic** 네트워크: `cas4160/networks/critics.py`에 있는 `ValueCritic`. 역시 관찰(observations)을 입력으로 받지만 가치 추정치(value estimates)를 출력한다. 이는 Section 5에서 구현해야 한다.

훈련 반복(iteration)은 `infrastructure/utils.py`에 정의된 `sample_trajectories()`를 사용하여 궤적(trajectories)을 샘플링하는 것으로 시작하며, `agents/pg_agent.py`에 있는 `PGAgent.update()`를 통해 에이전트를 업데이트한다.

이 `update()` 과정에서 `PGAgent`는 먼저 보상을 Q-값으로 처리하고(`PGAgent._calculate_q_vals()`), 그 다음 어드밴티지를 추정한다(`PGAgent._estimate_advantage()`). 이후 actor는 `cas4160/networks/policies.py`에 있는 `MLPPolicyPG.update()`를 통해, critic은 `cas4160/networks/critics.py`에 있는 `ValueCritic.update()`를 통해 각각 업데이트된다. 만약 `--use_ppo` 플래그가 설정되어 있다면, actor 업데이트 시 표준 `MLPPolicyPG.update()` 메서드 대신 `MLPPolicyPG.ppo.update()` 메서드가 사용된다. `MLPPolicyPG.ppo.update()` 메서드는 Section 7에서 작업하게 될 것이다.

4 Policy Gradients

4.1 Implementation

너는 `pg_agent.py` 내에서 두 가지 다른 리턴 추정기(return estimator)를 구현하게 될 것이다. 이 둘은 합산의 시작점만 다르다는 점에 유의하라.

첫 번째 방법("Case 1", `PGAgent._calculate_q_vals()` 내부)은 전체 궤적(full trajectory)의 할인된 누적 리턴(discounted cumulative return)을 사용하며, 이는 정책 경사의 "vanilla" 형태(Equation (2))에 해당한다:

$$r(\tau_i) = \sum_{t'=0}^{T-1} \gamma^{t'} r(\mathbf{s}_{it'}, \mathbf{a}_{it'}). \quad (15)$$

두 번째 방법("Case 2")은 Equation (3)의 "reward-to-go" 공식을 사용한다:

$$r(\tau_i) = \sum_{t'=t}^{T-1} \gamma^{t'-t} r(s_{it'}, \mathbf{a}_{it'}). \quad (16)$$

이 두 리턴 추정기와 코드에서 TODO로 표시된 나머지 부분들을 구현하라. 소규모 실험(small-scale experiments)의 경우, **nn.baseline is True**일 때만 실행되는 부분은 건너뛰어도 된다. 베이스라인(`MLPPolicyPG.update()`와 `PGAgent._estimate_advantage()`)은 Section 5에서 다시 다룰 것이다.

4.2 Experiments

Experiment 1 (CartPole). PG 알고리즘을 discrete CartPole-v0 환경에서 실행하여 여러 실험을 수행하라. 다음 명령어를 사용한다:

```
python cas4160/scripts/run_hw2.py --env_name CartPole-v0 -n 100 -b 1000 \
    --exp_name cartpole

python cas4160/scripts/run_hw2.py --env_name CartPole-v0 -n 100 -b 1000 \
    -rtg --exp_name cartpole_rtg

python cas4160/scripts/run_hw2.py --env_name CartPole-v0 -n 100 -b 1000 \
    -na --exp_name cartpole_na

python cas4160/scripts/run_hw2.py --env_name CartPole-v0 -n 100 -b 1000 \
    -rtg -na --exp_name cartpole_rtg_na

python cas4160/scripts/run_hw2.py --env_name CartPole-v0 -n 100 -b 4000 \
    --exp_name cartpole_lb

python cas4160/scripts/run_hw2.py --env_name CartPole-v0 -n 100 -b 4000 \
    -rtg --exp_name cartpole_lb_rtg

python cas4160/scripts/run_hw2.py --env_name CartPole-v0 -n 100 -b 4000 \
    -na --exp_name cartpole_lb_na

python cas4160/scripts/run_hw2.py --env_name CartPole-v0 -n 100 -b 4000 \
    -rtg -na --exp_name cartpole_lb_rtg_na
```

여기서 각 인자는 다음을 의미한다:

- **-n**: 반복 횟수(Number of iterations).
- **-b**: 배치 크기(Batch size). 각 반복에서 현재 정책에 따라 행동하면서 샘플링되는 상태-행동 쌍(state-action pairs)의 개수.
- **-rtg**: 플래그. 이 플래그가 있으면 `reward_to_go=True`로 설정된다. 없으면 기본적으로 `reward_to_go=False`가 된다.
- **-na**: 플래그. 이 플래그가 있으면 `normalize_advantages=True`로 설정되어, 배치 내 어드밴티지의 평균을 0, 표준편차를 1로 정규화(normalize)한다.

- `--exp_name` : 실험 이름. 데이터 로깅 디렉토리 이름에 포함된다. 실행이 성공하면, 텐서보드 로그 데이터는 다음 경로에서 찾을 수 있다:
`hw2_starter_code/data/q2.pg_[---exp_name]_[---env_name]_[current_time]/`.

그 외 다양한 명령줄 인자를 통해 배치 크기, 학습률(learning rate), 네트워크 구조 등을 설정할 수 있다. 모든 인자 목록은 `cas4160/scripts/run_hw2.py:main()`에서 확인할 수 있다.

참고: 정책 롤아웃(policy rollouts)의 비디오를 생성하려면 `--video_log_freq 10` 플래그를 추가하라. 이는 매 10 반복마다 평가 롤아웃을 로깅한다. 생성된 비디오는 텐서보드의 "Images" 탭에서 확인할 수 있다.

Deliverables for report:

- 두 개의 그래프를 생성하라:
- 첫 번째 그래프에서는 `cartpole` 점두사가 붙은 실험들(소규모 배치 실험)의 학습 곡선(learning curves)을 비교하라. (평균 리턴 vs. 환경 스텝 수)
- 두 번째 그래프에서는 `cartpole_lb` 점두사가 붙은 실험들(대규모 배치 실험)의 학습 곡선(learning curves)을 비교하라.

참고: 이 과제의 모든 그래프에서 x -축은 환경 스텝 수(number of environment steps)로 해야 하며, `Train.EnvstepsSoFar` 로 로깅된 값을 사용한다. (정책 경사 반복 횟수가 아니다.)

참고: 텐서보드 로그에서 데이터를 파싱하고 그래프를 그리는 데 예시 헬퍼 스크립트(`cas4160/scripts/parse_tensorboard.py`)를 사용할 수 있다. 다음은 그래프를 `output_plot.png`로 저장하는 예시 사용법이다:

```
python cas4160/scripts/parse_tensorboard.py \
--input_log_files data/[your_log_folder_1] data/[your_log_folder_2] \
--human_readable_names "Vanilla" "Reward to go" \
--data_key "Eval_AverageReturn" \
--title "Your Title Here" \
--x_label_name "Train Environment Steps" \
--y_label_name "Eval Return" \
--output_file "output_plot.png"
```

`--plot_mean_std` 인자를 추가하면, 지정한 `input_log_files`에 대해 표준편차(standard deviation)를 음영 처리된 영역(shaded area)으로 함께 그릴 수 있다. 시각화를 더 좋게 하기 위해 필요에 따라 이 파싱 스크립트를 수정하여 사용해도 된다.

What to Expect:

- 대규모 배치와 소규모 배치 경우 모두 `CartPole`의 최적 설정은 최대 점수 200에 수렴해야 한다. 또한, 비디오 로깅 없이 각 실험은 약 5분 정도 소요된다.

4.3 Policy Gradient Result

4.3.1 Plot (Eval Average Return with batch size 1000 and 4000):

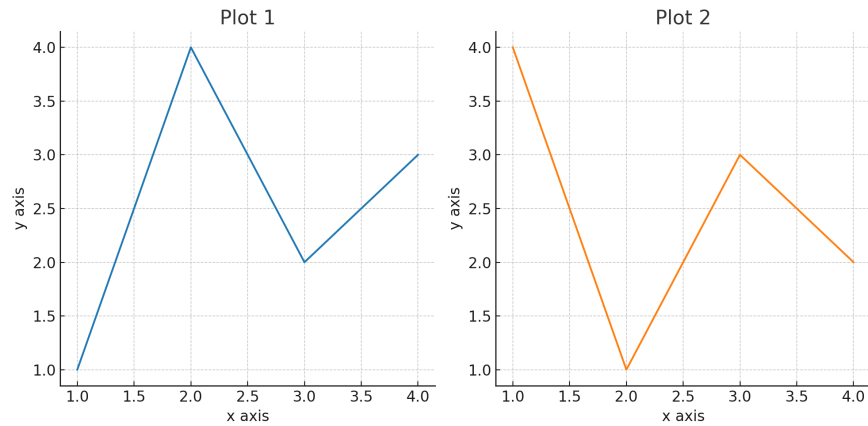


Figure 1: **Replace this with your plot, and add the caption.**

4.3.2 Questions:

Answer the following questions briefly:

- advantage normalization을 사용하지 않았을 때, 어떤 가치 추정기가 더 좋은 성능을 보이는가: 전체 궤적 보상의 합(REINFORCE)인가, 아니면 reward-to-go를 사용하는 것인가?
Answer: 여기에 답변을 작성하세요.
- advantage normalization이 도움이 되었는가?
Answer: 여기에 답변을 작성하세요.
- 배치 크기(batch size)가 영향을 미쳤는가?
Answer: 여기에 답변을 작성하세요.
- 실험을 실행할 때 사용한 정확한 명령줄 설정(command line configurations)을 제공하라. 기본값에서 변경한 모든 매개변수를 포함한다.
Answer: 여기에 답변을 작성하세요.

5 Using a Neural Network Baseline

5.1 Implementation

이제 상태에 의존하는 신경망 베이스라인(state-dependent neural network baseline)으로서 가치 함수(value function)를 구현할 것이다. 이는 4 섹션에서 건너뛴 몇 가지 TODO 부분을 채워야 한다. 특히 다음을 수행해야 한다:

- 이 신경망은 MLPPolicyPG의 update 메서드에서 정책 경사 업데이트와 함께 학습된다.
- PGAgent._estimate_advantage()에서는 이 네트워크의 예측값을 reward-to-go에서 빼서 어드밴티지(advantage)의 추정치를 얻는다. 이는 $\left(\sum_{t'=t}^{T-1} \gamma^{t'-t} r(s_{it'}, a_{it'})\right) - V_{\phi}^{\pi}(s_{it})$ 를 구현한 것이다.

- 각 정책 업데이트마다 베이스라인 네트워크를 `baseline_gradient_steps` 매개변수로 결정되는 여러 번의 그래디언트 스텝으로 학습한다.

5.2 Experiments

다음으로, 베이스라인을 적용한 정책 경사 구현을 사용하여 HalfCheetah-v4에 대한 컨트롤러(controller)를 학습할 것이다.

Run the following commands:

```
# No baseline
python cas4160/scripts/run_hw2.py --env_name HalfCheetah-v4 \
  -n 100 -b 5000 -rtg --discount 0.95 -lr 0.01 \
  --exp_name cheetah
# Baseline
python cas4160/scripts/run_hw2.py --env_name HalfCheetah-v4 \
  -n 100 -b 5000 -rtg --discount 0.95 -lr 0.01 \
  --use_baseline -blr 0.01 -bgs 5 --exp_name cheetah_baseline
```

너는 `-na` (normalize advantages)를 생략한 것을 눈치챘을 수도 있다. 이는 실제로 advantage normalization이 매우 강력한 기법이기 때문이며, 대부분의 간단한 환경에서는 베이스라인(baseline)이 필요 없게 만든다.

Deliverables:

- 베이스라인 손실(baseline loss)에 대한 학습 곡선(learning curve)을 그려라.
- 평가 리턴(eval return)에 대한 학습 곡선(learning curve)을 그려라.
- 베이스라인 그래디언트 스텝 수(`-bgs`)와/또는 베이스라인 학습률(`-blr`)을 줄인 또 다른 실험을 실행하라. 이것이 (a) 베이스라인 학습 곡선과 (b) 정책의 성능에 어떤 영향을 미치는지 설명하라.
- (선택사항) `-na`를 다시 추가하여 얼마나 성능이 향상되는지 확인하라. 또한 `--video_log_freq 10`을 설정한 후, TensorBoard를 열고 “Images” 탭으로 이동하여 HalfCheetah가 걸어가는 영상을 확인하라!

What to Expect:

- 베이스라인 버전에서는 평균 리턴이 300 이상이 되는 것을 기대할 수 있다. 또한, 비디오 로깅 없이 각 실행은 약 10분 정도 소요된다.

5.3 Neural Network Baseline Result

5.3.1 Plot (Baseline Loss, Eval Average Return):

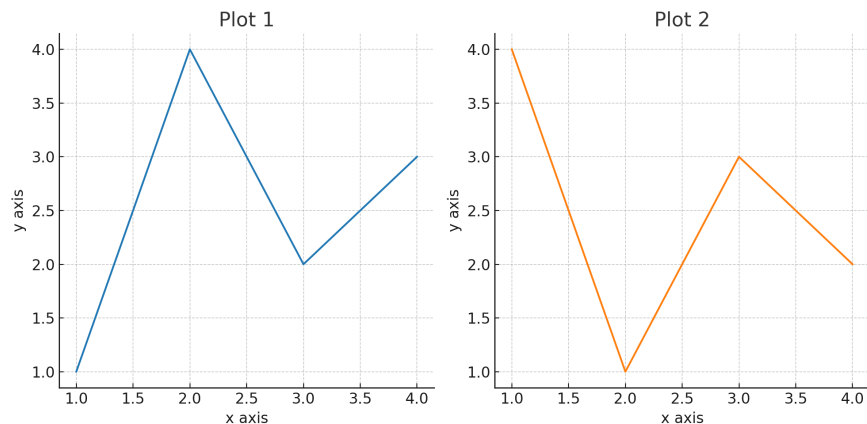


Figure 2: **Replace this with your plot, and add the caption.**

5.3.2 Questions:

Answer the following questions briefly:

- 베이스라인 그래디언트 스텝 수(-bgs)와/또는 베이스라인 학습률(-blr)을 줄인 또 다른 실험을 실행하라. 이것이 (a) 베이스라인 학습 곡선과 (b) 정책의 성능에 어떤 영향을 미치는지 설명하라. **Answer: Include your answer here.**

6 Generalized Advantage Estimator

6.1 Implementation

이제 이전에 구현한 가치 함수를 사용하여 GAE- λ 의 간소화된 버전을 구현할 것이다. 이는 `PGAgent._estimate_advantage()`에 남아 있는 TODO 섹션을 채우는 작업이 필요하다.

6.2 Experiments

Experiment 3 (HumanoidStandup-v5). 이제 Generalized Advantage Estimation을 적용한 정책 경사 구현을 사용하여 HumanoidStandup-v5 버전에 대한 컨트롤러를 학습할 것이다. 아래 $\langle \lambda \rangle$ 를 대체하기 위해 $\lambda \in [0, 0.95, 1]$ 범위에서 탐색하라. 다른 하이퍼파라미터(예: 배치 크기, 학습률 등)는 변경하지 마라.

```
python cas4160/scripts/run_hw2.py \  
  --env_name HumanoidStandup-v5 --ep_len 100 \  
  --discount 0.99 -n 50 -l 3 -s 128 -b 2000 -lr 0.001 \  
  --use_reward_to_go --use_baseline --gae_lambda <\lambda> \  
  --exp_name HumanoidStandup_lambda<\lambda>
```

Deliverables:

- 당신이 시도한 HumanoidStandup-v5 실험들의 학습 곡선(learning curves)을 하나의 그래프로 제공하라. λ 가 과제 성능(task performance)과 훈련 효율성(training efficiency)에 어떤 영향을 미쳤는지 글로 설명하라.
- 매개변수 λ 에 대해 고려하라. $\lambda = 0$ 은 무엇에 해당하는가? $\lambda = 1$ 은 무엇에 해당하는가? 이것을 HumanoidStandup-v5에서의 과제 성능과 관련지어 한두 문장으로 설명하라.

What to Expect:

- The run with the best performance should achieve an average score close to $1.15e4$. In addition, each run takes about 10 minutes without video logging.

6.3 GAE Result

6.3.1 Plot (Eval Average Return with different λ):

- 가장 좋은 성능을 보인 실험은 평균 점수가 1.15×10^4 에 가까워야 한다. 또한, 비디오 로깅 없이 각 실험은 약 10분 정도 소요된다.

6.3.2 Questions:

Answer the following questions briefly:

- λ 가 과제 성능(task performance)에 어떤 영향을 미쳤는지 글로 설명하라.
Answer: 여기에 답변을 작성하세요.
- 매개변수 λ 에 대해 고려하라. $\lambda = 0$ 은 무엇에 해당하는가? $\lambda = 1$ 은 무엇에 해당하는가? 이것을 HumanoidStandup-v5에서의 과제 성능과 관련지어 한두 문장으로 설명하라.
Answer: 여기에 답변을 작성하세요.

7 Proximal Policy Optimization

마침내 Proximal Policy Optimization (PPO) 알고리즘을 구현할 준비가 되었다. 이 알고리즘은 surrogate advantage를 계산하기 위해 generalized advantage estimate를 사용한다. PPO에는 PPO-Penalty와 PPO-Clip이라는 두 가지 주요 변형이 있다. 이번 섹션에서는 PPO-Clip을 구현할 것이며, 그 목적 함수(objective)는 다음과 같이 정의된다:

$$L(\mathbf{s}, \mathbf{a}, \theta_k, \theta) = \min \left(\frac{\pi_{\theta}(\mathbf{a} | \mathbf{s})}{\pi_{\theta_k}(\mathbf{a} | \mathbf{s})} A^{\pi_{\theta_k}}(\mathbf{s}, \mathbf{a}), \quad \text{clip} \left(\frac{\pi_{\theta}(\mathbf{a} | \mathbf{s})}{\pi_{\theta_k}(\mathbf{a} | \mathbf{s})}, 1 - \epsilon, 1 + \epsilon \right) A^{\pi_{\theta_k}}(\mathbf{s}, \mathbf{a}) \right),$$

여기서 θ_k 는 업데이트 이전의 이전 정책 파라미터(old policy parameters)를 의미한다.

주요 아이디어는 업데이트 후 새로운 정책이 이전 정책에서 너무 멀어지지 않도록 하는 것이다. 이를 위해 PPO는 너무 큰 업데이트를 방지하기 위해 clipping을 사용한다. 이러한 형태의 규제(regulation)를 통해 동일한 데이터로 여러 에포크(epoch)의 미니배치 업데이트(minibatch updates)를 수행할 수 있으며, 이는 더 나은 안정성(stability)과 샘플 효율성(sample efficiency)을 가져온다. PPO에 대해 더 알고 싶다면, 잘 설명된 자료를 참고하라: [OpenAI Spinning Up, PPO 논문].

7.1 Implementation

PPO를 구현하기 위해서는 `MLPPolicyPG.ppo.update()`와 `PGAgent.update()`에 있는 TODO 섹션을 채워야 한다. 손실(loss) 계산을 위해 위에서 정의한 surrogate objective를 참고하라.

7.2 Experiments

Experiment 4 (Reacher-v4). Generalized Advantage Estimation을 사용한 PPO 구현을 이용하여 Reacher-v4에 대한 컨트롤러를 학습하라. 다음 명령어를 실행하여 실험을 수행하라.

```
# Reacher (Baseline)
python cas4160/scripts/run_hw2.py \
    --env_name Reacher-v4 --ep_len 1000 \
    --discount 0.99 -n 100 -b 5000 -lr 0.003 \
    -na --use_reward_to_go --use_baseline --gae_lambda 0.97 \
    --exp_name reacher
# Reacher (PPO)
python cas4160/scripts/run_hw2.py \
    --env_name Reacher-v4 --ep_len 1000 \
    --discount 0.99 -n 100 -b 5000 -lr 0.003 \
    -na --use_reward_to_go --use_baseline --gae_lambda 0.97 \
    --use_ppo --n_ppo_epochs 4 --n_ppo_minibatches 4 \
    --exp_name reacher_ppo
```

Deliverables:

- 당신이 시도한 Reacher-v4 실험들의 학습 곡선(learning curves)을 하나의 그래프로 제공하라. PPO를 사용한 최종 에이전트와 사용하지 않은 경우의 성능을 비교하라.
- surrogate objective를 사용하지 않고 여러 번의 배치 업데이트를 수행하면 어떤 일이 발생하는가? 그 결과를 정당화하라.
- (선택사항) 위 질문을 확인하기 위해 “actor.ppo_update()”를 “actor.update()”로 변경하고 성능을 비교해볼 수 있다.

What to Expect:

- PPO를 사용한 실험은 올바르게 구현되었다면 평균 점수가 -10에 가까워야 한다(최소 -15 이상). 또한, 비디오 로깅 없이 각 실험은 약 14분 정도 소요된다.

7.3 PPO Result

7.3.1 Plot (Eval Average Return for PPO and the baseline):

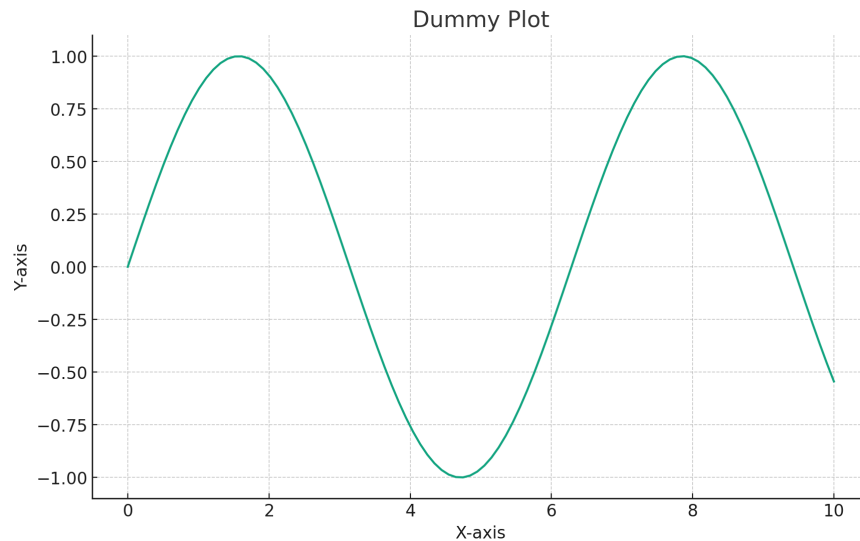


Figure 3: **Replace this with your plot, and add the caption.**

7.3.2 Questions:

Answer the following questions briefly:

- surrogate objective를 사용하지 않고 여러 번의 배치 업데이트를 수행하면 어떤 일이 발생하는가? 그 결과를 정당화하라.

Answer: 여기에 답변을 작성하세요.

8 Discussion

각 문제마다 대략적인 소요 시간을 시간 단위로 알려주십시오. 이는 향후 과제 난이도를 조정하는 데 도움이 됩니다.

- Policy Gradients: **XX hours**
- Neural Network Baseline: **XX hours**
- Generalized Advantage Estimator: **XX hours**
- Proximal Policy Optimization: **XX hours**

Feel free to share your feedback here, if any: **We would really appreciate your feedback to improve the reinforcement learning class.**

9 Submission

제출물에는 코드, tensorboard 로그 데이터, 그리고 **“report”**를 포함시켜 주십시오. 이를 hw2-[학번].zip으로 압축하라. 제출 파일의 구조는 다음과 같아야 한다:

```
hw2_[YourStudentID].zip
├─ hw2_[YourStudentID].pdf
├─ cas4160/
│   └─ ...codes
├─ data/
│   └─ ...tensorboard log folders
└─ ...
```

Note: Do NOT include the videos (.mp4 files or tensorboard logs) in your submission. Your submission file size should be **less than 50MB**.